



Programação Avançada 2025-26

**09 Grafos | Travessias**

Bruno Silva, Patrícia Macedo

# Sumário

- Travessias;
- Breadth-first e Depth-first;
- Exercícios.

# Grafos | Travessias

- As **estruturas lineares** são percorridas sequencialmente, normalmente de uma extremidade para a outra.
- As **estruturas não lineares** podem ser percorridas de diversas formas:
  - As árvores pode ser percorridas em:
    - **Largura**;
    - **Profundidade** (*pos-order* e *pre-order*).
  - Os grafos também podem ser percorridos em:
    - **Largura** (*breadth-first*);
    - **Profundidade** (*depth-first*).

⚠ Note que no caso dos grafos **não existe** um "ponto-de-partida" óbvio, e.g., a raiz de uma árvore.

# Grafos | Travessias

Para que servem? Apenas alguns exemplos:

- **Busca de Caminhos e Conectividade:** As travessias em grafos são usadas para encontrar caminhos entre dois nós, verificar a existência de caminhos ou determinar se o grafo é conectado. Isso é útil em sistemas de navegação, redes de computadores e análise de conectividade em redes sociais.
- **Exploração de Grafos para Análise de Redes Sociais:** Em redes sociais, as travessias são usadas para analisar a estrutura da rede, encontrar amigos em comum, identificar influenciadores, analisar tendências e detectar comunidades dentro da rede.
- **Base para outros algoritmos:** muitos outros algoritmos têm como base uma travessia de um grafo.

[https://en.wikipedia.org/wiki/Graph\\_traversal](https://en.wikipedia.org/wiki/Graph_traversal)

# Grafos | Travessias

Os algoritmos de **travessia de grafos** devem levar em conta a:

- **Eficiência:** um mesmo local não deve ser visitado mais do que uma vez.
  - 💡 Marcar os vértices já visitados.
- **Corretude:** o percurso deve ser feito de modo a que não se perca informação (onde estamos durante a travesia).
  - 💡 Manter uma *coleção* com todos os vértices ainda por visitar.

# Grafos | Breadth-first search (BFS)

Travessia em largura (exploração por níveis)

- Inicia-se num vértice (denominado *vértice raiz*) e **exploram-se todos os vértices adjacentes a este**;
- Para cada um dos vértices adjacentes, exploram-se os **seus vértices vizinhos** ainda não visitados, repetindo os passos até já não existirem mais vértices por visitar.
  - ⚠ Para tal usa-se uma estrutura com política de acesso **FIFO** (uma *queue* ! ) para guardar os vértices descobertos e ainda não visitados.

# Grafos | Breadth-first search (BFS)

Pseudo-código:

```
Algorithm: BFS(graph, vertice_root)
BEGIN
  setAsVisited(vertice_root)
  enqueue(q, vertice_root)
  WHILE q is not EMPTY
    v <- dequeue(q)
    process(v)
    FOR EACH w IN adjacents(v)
      IF w is not visited THEN
        setAsVisited(w)
        enqueue(q, w)
      END-IF
    END-FOR
  END-WHILE
END
```

# Grafos | Breadth-first search (BFS)

Possível visualizar múltiplos exemplos em:

<https://www.cs.usfca.edu/~galles/visualization/BFS.html>



# Grafos | Depth-first search (DFS)

Travessia em profundidade (exploram-se ramos completos)

- Inicia-se num vértice (denominado *vértice raiz*) e explora-se, tanto quanto possível, cada um dos seus ramos antes de retroceder.
  - Para tal usa-se uma estrutura com política de acesso **LIFO** (uma *stack* ! ) para guardar os nós descobertos e ainda não visitados.

# Grafos | Depth-first search (DFS)

Pseudo-código:

```
Algorithm: DFS(graph, vertice_root)
BEGIN
  setAsVisited(vertice_root)
  push(s, vertice_root)
  WHILE s is not EMPTY
    v <- pop(s)
    process(v)
    FOR EACH w IN adjacents(v)
      IF w is not visited THEN
        setAsVisited(w)
        push(s, w)
      END-IF
    END-FOR
  END-WHILE
END
```

⚠ Note que a estrutura é idêntica à do BFS, muda apenas o tipo de *coleção* usada.

# Grafos | Depth-first search (DFS)

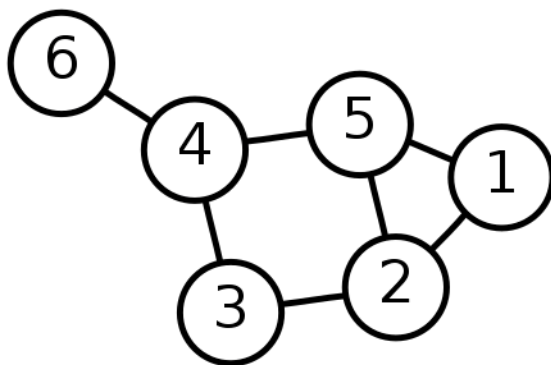
Possível visualizar múltiplos exemplos em:

<https://www.cs.usfca.edu/~galles/visualization/DFS.html>

# Grafos | Exercício

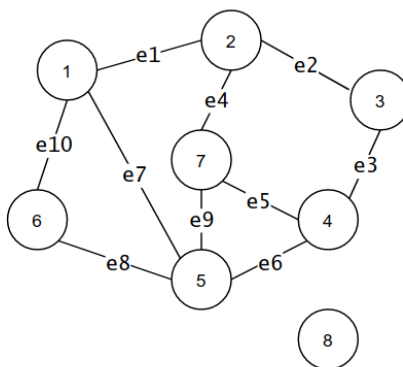
Percorra o grafo ilustrado com as travessias apresentadas, i.e., DFS e BFS.

- Mantenha um registo do estado dos *visitados* e *stack/queue*.
- Inicie as travessias no vértice 3.



# Treino em casa

Faça clone novamente do repositório: [https://github.com/estsetubal-pa/ADTGraph\\_Example\\_Template](https://github.com/estsetubal-pa/ADTGraph_Example_Template)



1. Implemente o método seguinte, que devolve a coleção de vértices segundo uma travessia *depth-first* (⚠ ver notas):

```
public static Collection<Vertex<Integer>> dfs(Graph<Integer, String> graph, Vertex<Integer> start) {  
    ...  
}
```

# Treino em casa

2. Teste o método no grafo, com ponto de partida no vértice 7.
3. Crie e implemente um método análogo para a travessia *breadth-first* e teste no grafo com ponto de partida no vértice 7.

## ⚠ Notas:

- Determinar se um vértice já foi visitado, ou não, pode ser simplesmente obtido através de uma coleção, e.g.,:

```
List<Vertex<Integer>> visited = new ArrayList<>();  
visited.add(v);  
if(visited.contains(w)) { ... }
```

- *Stacks* e *queues* já existem em java, e.g.:

```
Stack<Vertex<Integer>> stack = new Stack<>(); // push(), pop()  
Queue<Vertex<Integer>> queue = new LinkedList<>(); // offer(), poll()
```